Рекурсия и линейные структуры

Цель: реализовать рекурсивные функции и структуры данных, проверить корректность и сравнить затраты с эталонами. Замеры выполнены 27.10.2026.

1. Реализация и сложность

Объект	Реализация и стоимость
Факториал	База 0! = 1, переход n·factorial(n-1). O(n) вызовов и памяти стека.
Фибоначчи	Наивно: $\Theta(\varphi^n)$ вызовов. Мемоизация: Θ(n) вызовов и записей кэша; стек O(n).
Ханойские башни	$2^n - 1$ ходов; стек O(n). При записи ходов выход занимает $\Theta(2^n)$.
DynamicArray	Начальная ёмкость 4, рост ×2; append амортизированно O(1), отдельно до O(n). get/set/pop O(1). Сжатия нет.
Stack / Deque	Стек LIFO использует только публичный API массива. Дек - двусвязный список, четыре операции концов O(1).

Оценки рекурсии даны по числу вызовов; длинная арифметика Python имеет дополнительную стоимость. Массив и стек сохраняют выделенную ёмкость после pop.

2. Выигрыш мемоизации: пустой кэш на каждый n

n	F(n)	Наивные вызовы	С мемоизацией
5	5	15	9
10	55	177	19
15	610	1973	29
20	6765	21891	39
25	75025	242785	49
30	832040	2692537	59

$C_{naive}(n) = 2F(n+1) - 1$. $C_{memo}(0) = 1$; $C_{memo}(n) = 2n - 1$ при $n \geq 1$. Считаются все входы, включая попадания в кэш. На n = 30: 2 692 537 против 59 вызовов.

3. Верификация

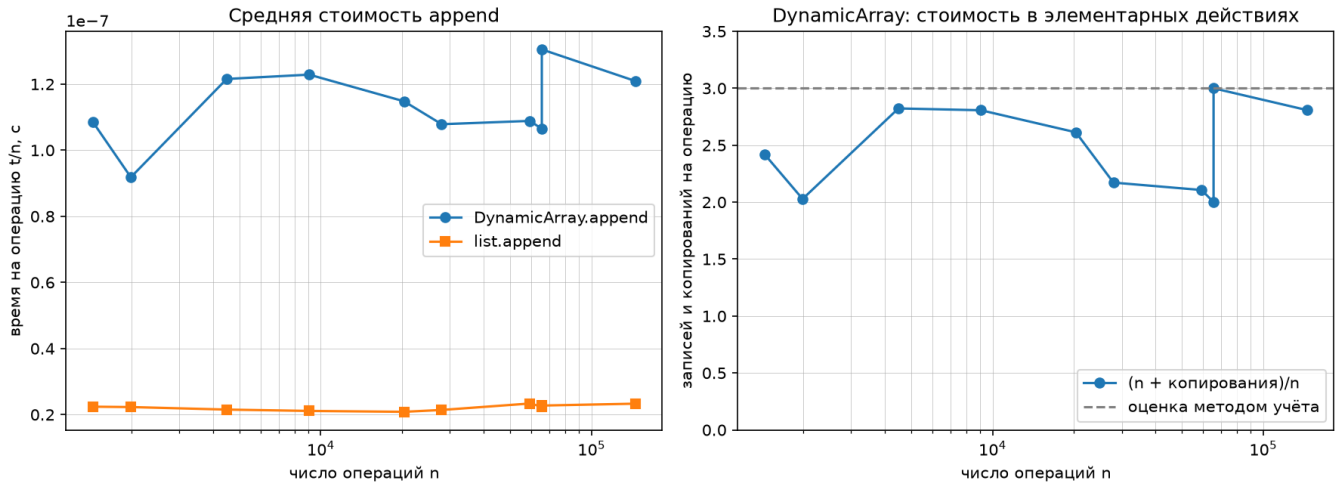
Пройдены все 5 разделов self_check заготовки, сверка 100 000 операций стека с list и 100 000 операций дека с collections.deque. Добавлены 900 случайных сценариев (seed 37), проверки ссылок двусвязного списка и очистки неактивных ячеек буфера. Проверки используют явные исключения и работают с python -O.

Факториал сверен с math.factorial на 0..20; Фибоначчи - с итеративным эталоном, мемоизация до n = 90. Ханой: n = 0, 1, 2, 3, 6, 7, проверка каждого хода и формулы. Black/Ruff пройдены. Повторная генерация данных совпала побайтно.

Амортизированная стоимость append

Один прогрев, 5 повторов, медиана perf_counter. Данные загружены до таймера. Время append делится на длину серии; создание пустого контейнера включено в запуск. Счётчик copies учитывает только перенос существующих элементов.

n	Ёмкость	copies	(n+copies)/n	Своя, мкс	list, мкс
1442	2048	2044	2.417476	0.108	0.022
1988	2048	2044	2.028169	0.092	0.022
4492	8192	8188	2.822796	0.122	0.022
9065	16384	16380	2.806950	0.123	0.021
20309	32768	32764	2.613275	0.115	0.021
27964	32768	32764	2.171649	0.108	0.021
59229	65536	65532	2.106417	0.109	0.023
65536	65536	65532	1.999939	0.106	0.022
65537	131072	131068	2.999908	0.130	0.023
144955	262144	262140	2.808423	0.121	0.023



Метод учёта: по 3 условные единицы за append

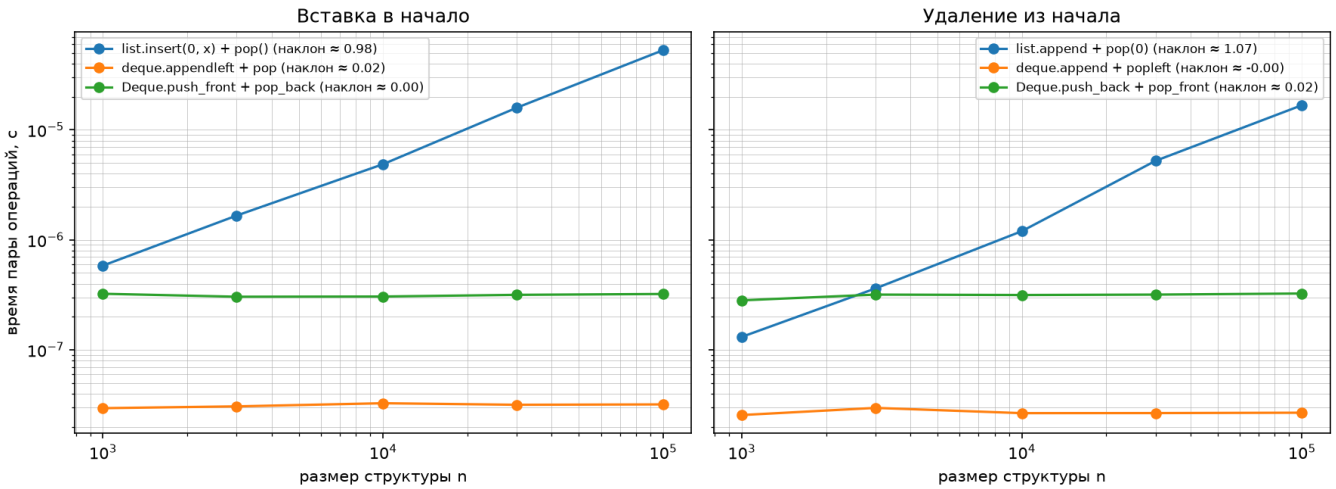
Одну единицу тратим на запись нового элемента, две сохраняем как кредит. Между расширениями C/2 → C и C → 2C происходит C/2 добавлений: они накопят C единиц для копирования C элементов. Первые 4 добавления также обеспечивают кредит для первого расширения.

Сумма копирований: 4 + 8 + ... + C/2 = C - 4. При n > 4 имеем C < 2n, поэтому (n + copies)/n < 3. На всех размерах варианта граница соблюдена. Инициализация нового буфера не входит в этот счётчик, но её геометрическая сумма также O(n).

На n = 65 536 стоимость 1,999939; следующий append вызывает рост и даёт 2,999908. Это допустимый скачок: отдельная операция O(n), а серия - O(n) суммарно и O(1) амортизированно на добавление.

Сравнение с list и collections.deque

Для каждого размера выполняется 1 000 пар «добавить + удалить»; размер структуры сохраняется. Подготовка заполненного контейнера исключена. На графике время пары: исследуемая операция у начала плюс O(1)-операция на другом конце.



Операция у начала	Наклон list	Наклон deque	Наклон своего дека
Вставка	0.980	0.017	0.003
Удаление	1.075	-0.001	0.025

У list вставка и удаление в начале требуют сдвига элементов: O(n), наклоны близки к 1. В collections.deque и своём деке число действий на концах не зависит от длины: O(1), наклоны близки к 0.

Последовательности операций варианта

100 000 операций	Своя, мкс/оп.	Эталон, мкс/оп.	Отношение
Stack / list	0.147	0.038	3.85×
Deque / collections.deque	0.201	0.039	5.18×

Свой стек и дек медленнее эталонов в постоянное число раз: методы и узлы реализованы на Python, а встроенные структуры - на C. У двусвязного списка есть расходы на отдельные узлы, две ссылки и худшую локальность памяти.

Вывод и воспроизводимость

Реализации принимаются по результатам проверок. Мемоизация устраняет повторные подзадачи; эксперимент и copies согласуются с амортизированной O(1) для append. Операции у начала list линейные, у обоих деков - постоянные по порядку роста.

Исходные 270 времён (54 точки × 5 повторов), хеши кода, генератора и входных файлов сохранены в measurements.json; сверки - в verification.json. Это один сеанс измерений; абсолютные времена зависят от условий запуска.

Источники: КИМ-02, заготовка и генератор курса (CC BY 4.0, МТУСИ). Часть загрузки, проверок и эксперимента адаптирована из заготовки; подробности в SOURCES.md.